

The Pillar Message Format: One Sealed, Content-Addressed Envelope for Every Byte Pillar Persists or Transmits

pillar engineering (machine-checked design note)

2026-09-09

Abstract

Pillar today carries what is fundamentally one thing—a signed, content-addressed record—in three unrelated byte formats: streamdb ops (hand-rolled length-prefixed `SignedSegment`, persisted to an embedded IPFS node), observability signals (an in-memory timeseries struct, never persisted or replicated), and libp2p control traffic (per-protocol CBOR, encrypted only by the transport’s Noise session). Three formats mean three serializers, three version stories, three security postures—and observability gets none of the durability, content-addressing, or replication streamdb already has. We unify them behind a single record: the `PillarMessage`, a version-stamped envelope whose payload is AEAD-sealed to the owning *cell* and addressed by the SHA2-256 multihash of its canonical CBOR encoding. The crux is a *convergent* content seal: the nonce is derived (HKDF) from the cell key and the content address of the plaintext, so identical bodies produce identical ciphertext and hence an identical `Cid` on every node—dedup, IPFS convergence, and Merkle roots all survive encryption—while distinct plaintexts never share a nonce (AEAD-safe). streamdb ops, observability signals, and every control message become body variants of the one envelope, owned by a new shared crate `pillar-wire` onto which streamdb, observability, and net all depend *down*. On the wire the envelope rides a pillar-UDP datagram sealed under a portable, cell-minted session key (companion paper), or a QUIC/TCP connection under the transport’s own TLS; the content seal is transport-agnostic and holds either way, so no transport ever sees plaintext. Per Pillar’s method #1, this is the design of record and the implementation is TLA⁺-gated: `OneRecordFormat`, `ContentAddressStable`, `DedupUnderEncryption`, `CellConfidential`, and `NoNonceReuseAcrossDistinctPlaintext` must be green under TLC before any Rust.

1 Motivation

The three formats Pillar uses today for one logical object—a signed, content-addressed record—are: **streamdb ops**, `SignedSegment{bytes,signer,signature,visibility}`, a hand-rolled length-prefixed wire form addressed

by `Cid=content_address(to_wire())` and persisted through `ContentStore/IpfsBackend` onto Pillar’s embedded IPFS node; **observability signals**, `Signal{id,kind,payload,labels,expiry}`, held only in an in-memory `TimeseriesStore`, never persisted, never on the wire; and **libp2p control traffic**, per-protocol CBOR request/response types plus gossipsub event-log messages, each its own struct, encrypted only by the libp2p Noise session at the transport layer.

Three formats mean three serializers, three version stories, three security postures, and—critically—observability signals get *none* of the durability, content-addressing, or replication that streamdb ops already have. The five signal kinds (metric/log/trace/profile/metadata) are already specified to ride “IPFS (durable signed segments/backfill) + streaming-DB tip ... no parallel store, no second authority” (ROI Priority 3). This paper makes that literal by giving signals the *same* record type streamdb ops use.

Thesis. There is exactly one Pillar record on the wire and on disk: the `PillarMessage`, a version-stamped envelope carrying a sealed, content-addressed payload, of which streamdb ops, observability signals, and every control message are variants. It is (1) **content-addressed**—its `Cid` is the SHA2-256 multihash of its canonical encoding, identical on every node; (2) **sealed to the cell**—the payload is AEAD-encrypted under the cell group key, *convergently* so the `Cid` is stable across nodes despite encryption; (3) **version-stamped**—an independently incrementable surface version, per the ROI versioning spine; and (4) **the payload of a pillar-UDP datagram** (fallback QUIC, then TCP+TLS).

2 Two seals, deliberately

The design has two distinct seals because they protect different things (Table 1). The **content seal** is on `PillarMessage.body`, keyed by the cell group key with a *deterministic* (convergent) nonce; it gives confidentiality at rest, a stable `Cid`, and dedup. The **transport frame seal** is on the pillar-UDP datagram, keyed by a portable cell-minted session key with a collision-free nonce; it protects on-wire framing. On QUIC/TCP the transport’s native TLS plays the second role instead.

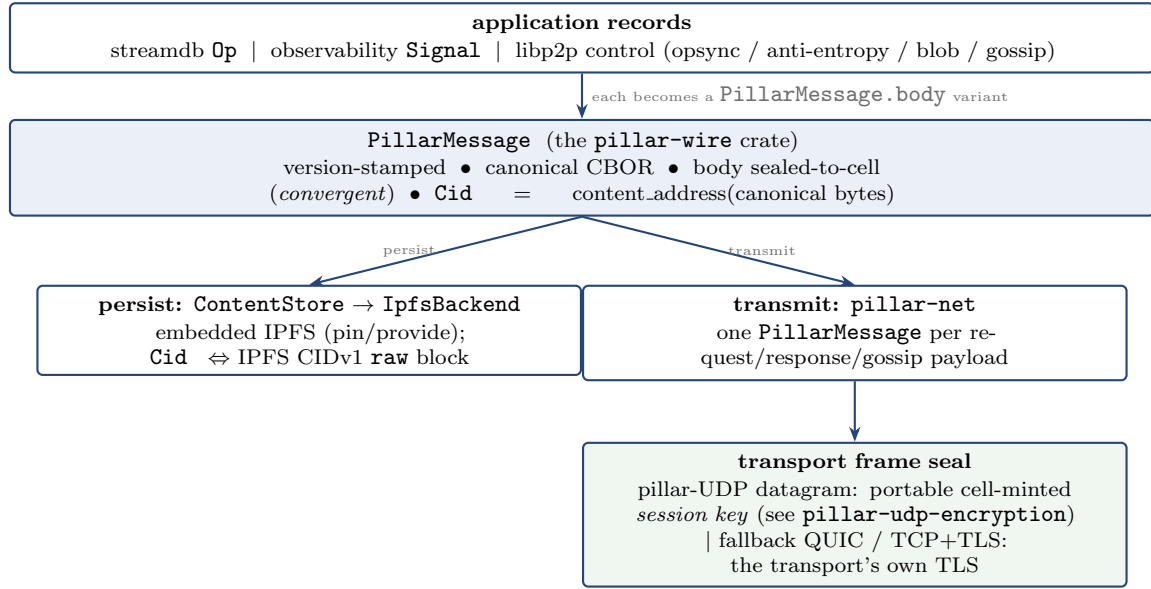


Figure 1: Layering. Every application record becomes the body of one `PillarMessage`; the envelope is content-addressed and cell-sealed once, then either persisted as an IPFS block or transmitted by `pillar-net`. The body seal travels *with* the record (an on-disk block is already sealed; a bitswap peer outside the cell cannot read it), so it is independent of, and composes with, the transport frame seal below—which is the pillar-UDP session key on pillar-UDP and the transport’s native TLS on QUIC/TCP. A datagram to a cell therefore carries a cell-sealed body inside a session-key-sealed datagram: double-sealed, each layer independent.

	content seal	transport frame seal
layer	<code>PillarMessage.body</code>	pillar-UDP datagram
recipient	the cell	the cell (any member)
key	cell group key (symmetric)	portable cell-minted session key K_s
nonce	deterministic (convergent)	collision-free (convergent / sender-partitioned)
purpose	confidentiality at rest + stable Cid + dedup	confidentiality of transport framing

Table 1: The two seals protect different things and compose. The content seal is transport-agnostic and travels with the record; the transport frame seal is pillar-UDP-specific (its portable session key is the subject of the companion paper `pillar-udp-encryption`). On QUIC/TCP the transport’s TLS supplies the frame seal instead.

3 pillar-wire: the shared substrate crate

Both `streamdb` and `observability` **depend down** onto one shared crate rather than sideways onto each other; that crate is `pillar-wire`—the common home for the wire/persistence primitives currently trapped inside `pillar-streamdb` (named because the *same* bytes go on the wire and to disk; the deliberately broad “widely used substrate” crate preferred over a scatter of opaque micro-crates). It owns `PillarMessage` (the envelope enum + its canonical CBOR codec + Cid derivation), the content seal (`cell_seal_convergent/cell_open_convergent`,

wrapping `pillar-crypto`), and—*moved down* from `pillar-streamdb`— `SignedSegment`, `Cid`, `HeadRecord`, `Visibility`, `ContentStore`, and the `IpfsBackend` trait. The dependency direction is strictly downward, no cycles:

`pillar-streamdb`, `pillar-observability`, `pillar-net` → `pillar-wire` → `pillar-crypto`, `pillar-core`, `pillar-ipfs`.

`pillar-wire` must *not* depend on `pillar-net` (net uses the envelope, not the reverse) nor on the two crates that depend on it.

4 The envelope

The one record—persisted and transmitted—and the body it seals:

```

struct PillarMessage {
    version      : SurfaceVersion
    signer       : SigningPublicKey
    signature    : Signature // over body_sealed
    visibility   : Visibility // DHT reach
    cell         : CellId // which group key
    opens it
    body_sealed : Ciphertext // convergent AEAD
}

enum Body { // what body_sealed decodes to
    StreamOp(StreamOpBody), // CRDT op-log
    Signal(SignalBody), // one of 5 kinds
    Control(ControlBody), //
    opsync/AE/blob/gossip
}
  
```

Canonical encoding. The envelope and Body are encoded with **deterministic CBOR** (RFC 8949 §4.2.1: sorted map keys, definite lengths, shortest-int)—CBOR because `pillar-net` already uses `request_response::cbor`, and determinism because the `Cid` is a hash of these bytes, so two nodes MUST produce byte-identical encodings for one logical record or content-addressing breaks. This replaces streamdb’s hand-rolled `to_wire`; `SignedSegment::to_wire/from_wire` become the `version=1` read-compatibility path (§7).

Content address. `Cid` = `content_address(canonical_cbor(PillarMessage))`, the existing `pillar-crypto` SHA2-256 multihash—the same function streamdb and observability already share, and the same multihash inside an IPFS CIDv1 raw block. Signing is over `body_sealed` (the ciphertext), so signature verification needs no cell key, decryption needs the cell key, and both are independent of the `Cid`.

5 Convergent content seal (the crux)

The promise is: *the encryption recipient is the cell; any cell node can decrypt; the Cid is the same across nodes; we deliver BOTH dedup AND encryption*. Naïvely this is impossible—`pillar-crypto::seal_symmetric` draws a random nonce, so the same signal sealed on two nodes yields different ciphertext, a different `Cid`, and no dedup. We therefore add a *convergent* seal:

```
nonce = HKDF(key=cell_group_key,
             info=".../convergent-nonce/v1" || domain,
             ikm=content_address(plaintext))[..n]
ct = AEAD_seal(cell_group_key, nonce, plaintext,
               aad=envelope_header)
```

Its properties: **deterministic**—the nonce is a pure function of (cell key, plaintext), so identical body + identical cell key $\Rightarrow$ identical ciphertext $\Rightarrow$ identical `Cid` on every node (the streamdb CRDT idempotence guarantee, now for sealed content); **confidential**—recovered only with the cell group key, itself sealed to each member by the existing `distribute_group_key` (X25519 sealed-box), so a non-member or bitswap peer holds only ciphertext; **nonce-safe**—reuse happens *only* for identical plaintext (which produces identical ciphertext anyway—the intended dedup), never across distinct plaintexts, because the nonce binds `content_address(plaintext)`; and an **equality leak**, accepted and documented—convergent encryption reveals that two sealed records are byte-identical (that is *how* dedup works), a property scoped to the cell confidentiality boundary, with the `domain` input separating record classes so equality never leaks across

classes. This is the one genuinely new cryptographic primitive the refactor introduces, and it earns its own TLC obligations and `pillar-crypto` contract tests.

6 Transport

The envelope’s on-wire encryption depends on the transport, and there are exactly two cases. **pillar-UDP** (a bare datagram substrate) supplies its own frame encryption keyed by a **portable, cell-minted session key**—established by a *cell-as-KDC* scheme distributed over streamdb, with the session key derived deterministically from the client’s ephemeral key against the cell static key. It replaces the libp2p Noise upgrade; it is neither handshakeless-per-peer nor a per-pair Noise session—the key is portable across every cell node, so any ingress/relay can carry the session. The complete scheme (packet flow, key derivation, anonymous handling, revocation/GC, the forward-secrecy tradeoff) is its own design of record, the companion paper `pillar-udp-encryption`, TLA⁺-gated by `specs/PillarUdpEncryption.tla`. **QUIC/TCP** (the fallbacks) bring their own **TLS 1.3**; the pillar-UDP scheme does not apply there. The selection order is pillar-UDP $\rightarrow$ QUIC $\rightarrow$ TCP+TLS, and because the body is *already* cell-sealed (§5), even the fallback transports never see plaintext application content. All libp2p control messages ride *inside* the envelope, exactly like ops and signals; pillar-UDP sits beneath IPFS in the stack. Removing the Noise upgrade is a deliberate, *breaking* change, safe only behind the versioning spine (§7).

7 Compatibility & versioning

Per the ROI versioning spine (independent per-surface stamps, $N-1$ window): the **envelope version** (`PillarMessage.version`) is `v1`= the legacy `SignedSegment` length-prefixed form (read-compat), `v2`= this canonical-CBOR convergent envelope; the **pillar-UDP protocol version** bumps for the Noise $\rightarrow$ session-key cutover; and the **sealed-artifact version** is the convergent-seal algorithm tag (its KDF carries a `"/v1"` info so a future scheme coexists). A stamped-but-unknown *future* version is rejected *distinctly* from a parse error, so a node one version ahead is refused cleanly, not silently mis-read. Migration is read-through: a `v1` `Cid` stays valid, new writes are `v2`, and the content-addressed store lets both coexist by construction—no history rewrite.

8 What changes, concretely

New crate `pillar-wire` (envelope + codec + convergent seal + the primitives moved

Invariant	Guarantee
OneRecordFormat	every persisted/transmitted byte is a PillarMessage
ContentAddress-Stable	Cid is a deterministic function of the logical record, equal across nodes despite sealing
DedupUnder-Encryption	identical sealed records collapse to one Cid /one block
CellConfidential	only a cell-key holder recovers Body ; a non-member/bitswap peer gets ciphertext only
NoNonceReuseAcross-DistinctPlaintext	convergent content nonces collide only for identical plaintext
Negotiation-Refuses-Incompatible, RollingCoexistence	envelope, pillar-UDP, and seal versions bump independently; a mixed-version (incl. Noise-era) swarm never mis-frames or partitions

Table 2: TLA⁺ obligations of `PillarMessageFormat.tla` (the transport-agnostic envelope + convergent content seal + version negotiation).

down from `streamdb`). `pillar-crypto` gains `cell_seal_convergent/cell_open_convergent` alongside the existing random-nonce `seal_symmetric`. `pillar-streamdb` `Op/OpLog` payloads become `StreamOp` bodies; persistence re-exports `pillar-wire`’s `ContentStore` (no CRDT/Merkle change). `pillar-observability` `Signals` become `Signal` bodies, each persisted as a sealed, content-addressed IPFS block via `ContentStore`, with cross-node backfill on the same substrate as `streamdb`. `pillar-net` makes every payload a **PillarMessage**, drops the pillar-UDP Noise upgrade for the portable session key, and turns the per-protocol CBOR types into `ControlBody` variants.

9 Invariants (before any Rust)

The design is TLA⁺-gated (method #1). It refines/extends the existing `versioning-compat-migration-spec` and `ObsIngestionSubstrate`; Table 2 lists the envelope/content obligations, all green under TLC in `specs/PillarMessageFormat.tla`. The transport-frame obligations (portable session-key convergence, cell-signed session records, anonymous-as-unattested policy, replay-via-dedup, revoked-key erasure) are proven separately in `specs/PillarUdpEncryption.tla`.

10 Resolved design points

Anonymous-session transport crypto—resolved: anonymous clients use the *exact same* pillar-UDP session-key scheme; anonymity is a *policy* property, not a crypto scheme (an anonymous key is cryptographically equivalent to a user/node key and subject to the same WoT/RBAC checks; lacking attestations,

the cell withholds sensitive data and privileged operations by default-deny). **Transport forward secrecy—accepted tradeoff:** the portable session key is derivable under the cell static key, so transport FS bounds to cell-key security plus erase-on-revoke—the same price the content seal already pays, with per-session ephemeral FS deliberately traded for cross-node portability. **Convergent-encryption equality leak (§5)**—accepted as the mechanism of dedup, scoped to the cell boundary, flagged so it is an explicit reviewed acceptance rather than an accident. The first two are detailed in `pillar-udp-encryption`.

References

- [1] C. Bormann and P. Hoffman, *Concise Binary Object Representation (CBOR)*, RFC 8949, 2020.
- [2] J. Benet, *IPFS—Content Addressed, Versioned, P2P File System*, arXiv:1407.3561, 2014.
- [3] H. Krawczyk, *Cryptographic Extraction and Key Derivation: The HKDF Scheme*, CRYPTO, 2010.
- [4] L. Lamport, *Specifying Systems*, Addison-Wesley, 2002.